ADVENTIST UNIVERSITY OF CENTRAL AFRICA (AUCA)

Faculty of Information Technology

WEB TECHNOLOGIES AND API SERVICES

Final Project Report

Production Management System

AI Edition (v2.0)

REST API integration, machine-learning models and real-time predictions

Case study: Le Mieux Ltd

Item	Details
Course	Web Technologies and API Services
Lecturer	Kundan Kumar
Technologies	Python, Django, Django REST-style API, scikit-learn (back end); HTML, CSS, Bootstrap, AdminLTE, Chart.js (front end)
Group	Group 13
Submission	October 2026

Group Members

No.	Full name	Registration number
1	Iradukunda Muhire Bertin	20251MBI046
2	Isaac Nkusi	20251MBI020
3	Rwabukanga Jean de Dieu	20251MBI010

Table of Contents

Executive Summary **4**
1. Introduction **5**
1.1 Background 5
1.2 Problem Statement 5
1.3 Objectives 5
1.4 Scope 6
2. System Overview and Architecture **7**
2.1 Technology Stack 7
2.2 Architecture 7
2.3 Project Structure 8
2.4 Users, Roles and Permissions 8
3. Data Layer **9**
3.1 New Models 9
3.2 Data Sources 9
3.3 Synthetic Data Generation 9
3.4 Dataset Overview 11
4. REST API **12**
4.1 Design Principles 12
4.2 Authentication and Authorisation 12
4.3 Endpoint Reference 12
4.4 Example: Ingesting an Incoming Order 13
4.5 Open API Integration (Exchange Rates) 13
5. Machine Learning **15**
5.1 Problem Formulation 15
5.2 Data Acquisition through the API 15
5.3 Feature Engineering 15
5.4 Preventing Data Leakage 16
5.5 Evaluation Method 16
5.6 Algorithm Comparison 16
5.7 Results of the Deployed Models 17
5.8 What the Models Learned 17
5.9 Limitations of the Models 18
6. Deployment in Django **19**
6.1 Model Registry and Artefacts 19
6.2 Inference Service 19
6.3 Self-Healing 19
6.4 Continuous Learning 19
6.5 Retraining 19
7. Real-Time Prediction of Incoming Data **20**
7.1 The Ingestion Pipeline 20
7.2 Risk Levels 20
7.3 Artisan Recommendation 20
7.4 Stock Check and Recommendations 21
7.5 Scoring Open Production Tasks 21
7.6 From Prediction to Action 21
7.7 Simulating a Stream of Orders 21
8. User Interface and Dashboard **22**
8.1 Enhanced Dashboard 22
8.2 AI Insights and Orders 23

8.3 Order Detail 24
8.4 What-if Predictor 25
8.5 Model Center 26
8.6 Task Detail: AI Risk Assessment 27
8.7 Navigation, Alerts and Dark Mode 27
9. Testing and Quality Assurance 29
9.1 Automated Tests 29
9.2 Manual Verification 29
9.3 Defects Found and Fixed During Development 29
10. Changes to the Existing System 31
10.1 Corrections to Version 1.0 31
10.2 Data Added to the Database 31
11. Installation and Operation Guide 32
11.1 Contents of the Delivered Folder 32
11.2 Requirements 32
11.3 Installation Steps 32
11.4 Optional Commands 32
11.5 Demonstration Script (about 5 minutes) 33
11.6 Configuration 33
11.7 Troubleshooting 33
12. Security Considerations 35
13. Limitations and Future Work 36
14. Conclusion 37
Appendix A. New Files Inventory 38
Appendix B. API Usage from Python 39
Appendix C. Glossary 40

Executive Summary

In the midterm project, Group 13 built the **Production Management System** for Le Mieux Ltd, a leather manufacturer that produces shoes, belts, bags and folders. That system brought production tasks, warehouse stock and human resources into one Django application and enforced the company's business rules: a task is accepted only if the warehouse holds enough material, stock is deducted and refunded automatically, artisans on leave cannot be assigned, and every task follows a strict state machine.

This final project extends the system from a **record-keeping tool** into a **decision-support tool**. Managers no longer only see what is happening; they are told what is likely to happen and what to do about it. The extension was delivered as a new Django app, `analytics`, without removing or weakening any original feature. Its main components are:

- **A REST API (version 1)** of 16 endpoints (18 operations) that exposes the catalogue, stock, artisans, tasks, training data, incoming orders, predictions and model information as JSON. It supports API-key authentication for machines and session authentication for the browser, with read access for every user and write access for managers only.
- **A training dataset** of 2,003 finished production batches: 2,000 synthetic batches generated from the company's *real* products, Bills of Materials, artisans and contracts, plus the real completed tasks. Every future completed task is added automatically.
- **Two machine-learning models trained on data read through the API.** A delay-risk classifier predicts the probability that a batch will finish after its due date (ROC-AUC **0.934** on unseen data) and a duration regressor predicts the number of days a batch will take (R^2 **0.893**, mean absolute error **1.70 days**). For each model three algorithms were compared and the best one (Gradient Boosting in both cases) was deployed.
- **Deployment inside Django** with a versioned model registry, an in-memory inference service and a self-healing mechanism that retrains the models automatically when the model file is missing or when the computer has a different scikit-learn version.
- **Real-time scoring of new incoming data.** Every new order that arrives through the API, the simulator or the web form is scored on arrival: delay risk, expected duration, forecast completion date, the best available artisan, a stock check and plain-language recommendations.
- **An enhanced dashboard** with new KPIs, a delay-risk monitor for open tasks, a live order feed that refreshes itself, a material demand forecast and three charts, plus five new pages (AI Insights, What-if Predictor, Model Center, Production History and API Center).
- **An Open API integration** that reads the live USD→RWF exchange rate to value the inventory in US dollars, with caching and an offline fallback so the ERP never depends on the internet.

Quality was verified with **40 automated tests**, including regression tests proving that the original business rules still behave exactly as before, and with a manual check of every page under the three user roles. Four small defects of the original system were also corrected, the most important being that an overdue task could no longer have its progress updated. All changes are documented in this report and in `docs/CHANGES.md`.

1. Introduction

1.1 Background

Le Mieux Ltd manufactures handcrafted leather goods. The midterm system (version 1.0) replaced paper records and separate spreadsheets with a single web application organised in three Django apps: `inventory` (raw materials, products and their Bills of Materials), `production` (production tasks and the dashboard) and `employees` (departments, artisans, contracts and leave). It calculates the production cost and profit margin of each product, protects the warehouse from over-commitment, and gives managers a dashboard of recent tasks, overdue alerts and artisan workload.

1.2 Problem Statement

Version 1.0 answers the question *"what is the situation now?"* but not *"what will happen next?"*. In daily operation the production manager still has to guess:

- whether a new order can realistically be delivered by the date the customer wants;
- which artisan should take a job so that it is finished on time;
- which open tasks are drifting towards a delay before they become overdue;
- which raw materials will run short if all pending orders are accepted;
- how other systems (a sales channel, a workshop terminal, a partner) can exchange data with the ERP automatically.

1.3 Objectives

The extension had ten requirements. Table 1 maps each requirement to the way it was fulfilled and where it can be seen in the system.

Table 1: *Requirements traceability*

#	Requirement	How it is fulfilled	Where to see it
1	Read the data from an API (open, registered or own Django API)	Own REST API built with Django (`/api/v1/`). The training pipeline reads its data from `GET /api/v1/records/`, in-process or over HTTP. An Open API (exchange rates) is also consumed.	Ch. 4; `train_models`; API Center
2	Train the model using original or synthetic data	2,000 synthetic batches generated from the real catalogue + real completed tasks. Two models, three algorithms compared for each.	Ch. 3 and 5; Model Center
3	Deploy the model in Django	Model registry (`MLModelVersion`), joblib artefacts, cached `PredictorService`, self-healing retraining, retraining from UI/CLI/API.	Ch. 6; Model Center
4	Read new incoming data, predict and display on the dashboard; improve the dashboard	`POST /api/v1/orders/` ingests and scores orders instantly; simulator; live feed refreshes every 15 s; new KPIs, risk monitor, forecast and charts.	Ch. 7 and 8; Dashboard
5	Add the functionality without breaking the current working principle	Additive design (new app, new tables only); fail-safe dashboard integration; regression tests of all original rules.	Ch. 9 and 10
6	Detailed report, ready for submission	This document.	—
7	Error-free, maximum	40 automated tests, manual role-based	Ch. 8 and 9

#	Requirement	How it is fulfilled	Where to see it
	functionality, professional UX	checks, consistent AdminLTE design, dark mode, clear empty/error states.	
8	Report everything needed to run the project	Installation guide, credentials, commands, demo script, configuration and troubleshooting.	Ch. 11
9	Add useful extras and document them without breaking anything	What-if Predictor, artisan recommendation, material demand forecast, continuous learning, live alert bell, task risk card, API key management, audit log.	Ch. 7, 8 and 10
10	Provide a downloadable zipped folder	Complete project with database, trained models, documentation and screenshots.	Ch. 11.1

1.4 Scope

The extension covers production planning decisions: delivery risk, duration, artisan assignment and material demand. It does not change the HR, costing or inventory rules of version 1.0, and it does not replace the manager's judgement: every prediction is presented with its reasons and recommendations, and nothing is created automatically without a manager's confirmation.

2. System Overview and Architecture

2.1 Technology Stack

Table 2: *Technology stack*

Layer	Technology	Role
Back end	Python 3.12+, Django 6.1	Web framework, ORM, authentication, admin
Machine learning	scikit-learn ≥ 1.6, NumPy, SciPy, joblib	Model training, evaluation and persistence
Front end	Bootstrap 4, AdminLTE 3, Font Awesome, jQuery	Responsive layout and components (unchanged from v1.0)
Charts and tables	Chart.js 2.9, DataTables	Dashboard charts, sortable/searchable tables
Database	SQLite	Same database file as v1.0, with 6 new tables
External data	ExchangeRate-API open endpoint (open.er-api.com)	Live USD→RWF rate (no key required)

2.2 Architecture

Figure 1 shows the architecture. The three original apps are untouched in their structure. The new `analytics` app sits beside them and is made of three layers: the **API layer** (authentication, serialisation, endpoints), the **ML layer** (features, synthetic data, training, inference and descriptive analytics) and an **integration layer** (a signal, a context processor and template tags) that connects the AI to the existing pages without rewriting them.

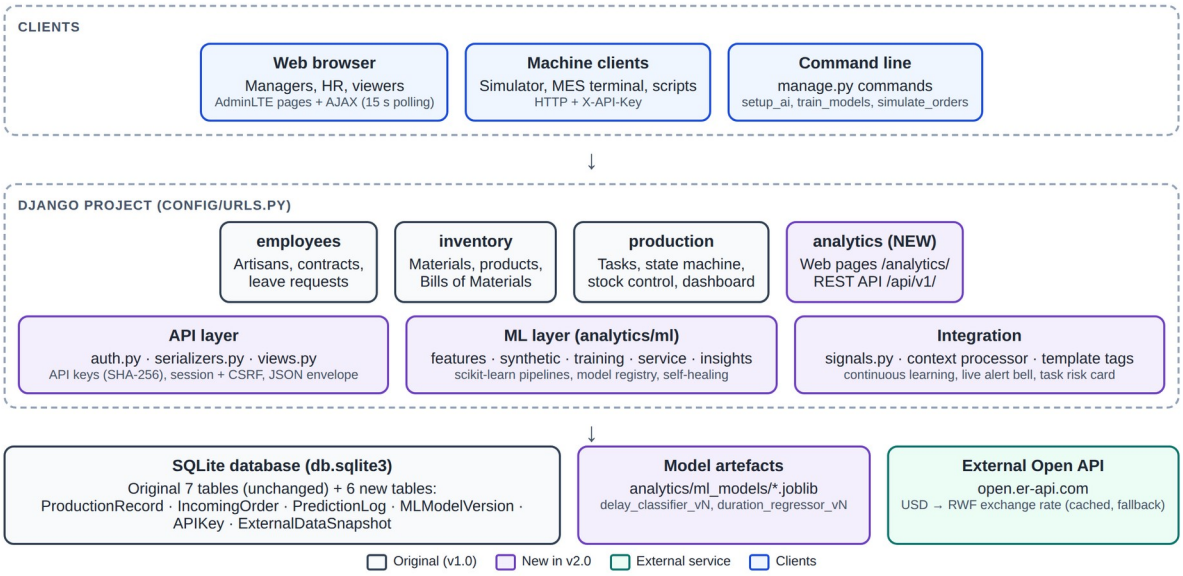


Figure 1: *System architecture of version 2.0 (purple = new components)*

Three kinds of clients use the system: people through the web browser, machine clients (the order simulator, a workshop terminal or any external program) through the REST API with an API key, and administrators through `manage.py` commands. All of them reach the same business logic, so an order sent by a script and an order typed in the web form are validated and scored in exactly the same way.

2.3 Project Structure

```
production_management/
├── config/              settings.py, urls.py (2 lines added each)
├── employees/           unchanged
├── inventory/           unchanged
├── production/          views.py (dashboard hook), models.py (bug fix), task_detail.html (AI
card)
├── analytics/           NEW app
│   ├── api/             auth.py, serializers.py, views.py, urls.py
│   ├── ml/              features.py, synthetic.py, training.py, service.py, insights.py,
external.py
│   ├── management/commands/  setup_ai, generate_synthetic_data, train_models,
│   │                         simulate_orders, create_api_key
│   ├── ml_models/       delay_classifier_vN.joblib, duration_regressor_vN.joblib
│   ├── templates/analytics/  insights, order_detail, predictor, model_center,
│   │                         history, api_center, _prediction_result
│   ├── static/analytics/analytics.css
│   ├── templatetags/analytics_tags.py
│   ├── models.py, views.py, urls.py, forms.py, admin.py
│   ├── signals.py, context_processors.py, dashboard.py
│   └── tests.py         40 automated tests
├── templates/           base.html (navigation), dashboard.html (new widgets)
├── docs/                this report, CHANGES.md, screenshots
├── backups/             database before the extension
├── db.sqlite3, requirements.txt, readme.md, Credentials, manage.py
```

2.4 Users, Roles and Permissions

The extension reuses the three roles of version 1.0. Table 3 summarises what each role can do with the new features; the original permissions are unchanged.

Table 3: *Permissions of the new features by role*

Capability	Viewer (User)	Manager (HR1)	Admin
View dashboard, AI Insights, Model Center, History, API docs	Yes	Yes	Yes
Run What-if predictions (web and `POST /predict/`)	Yes	Yes	Yes
Read API endpoints with own session or key	Yes	Yes	Yes
Submit / simulate incoming orders	No	Yes	Yes
Convert an order into a production task, dismiss an order	No	Yes	Yes
Retrain models (web, CLI or API)	No	Yes	Yes
Create and revoke API keys	No	Yes	Yes
Django Admin for the new models	No	If granted	Yes

3. Data Layer

3.1 New Models

Six models were added (Table 4). None of them changes an existing table. Every link to an original model uses `on_delete=SET_NULL`, so deleting a product, artisan or task can never fail because of analytics data, and history is preserved through snapshot fields.

Table 4: *New data models*

Model	Purpose	Key fields
ProductionRecord	One finished batch: the training data	snapshot of product (category, BOM size, price, material cost), artisan (contract, department, workload), stock coverage, start/due/end dates, actual_days, delayed, defective_units, source (synthetic / live / api)
IncomingOrder	A new production request and its prediction	reference, customer, channel, product, quantity, due_date, requested_artisan, delay_probability, risk_level, predicted_days, predicted_completion, recommended_artisan, stock_ok, prediction (JSON), status (New / Converted / Dismissed), converted_task
PredictionLog	Audit trail of every prediction served	kind (order / task / whatif / api), object_ref, inputs, outputs, model_version, user, created_at
MLModelVersion	Model registry	name, version, algorithm, file_name, metrics, candidates, feature_importances, n_train, n_test, data_source, sklearn_version, is_active, trained_by
APIKey	Credentials for machine clients	name, user, prefix, hashed_key (SHA-256), is_active, last_used_at
ExternalDataSnapshot	Last good value from the Open API	key, payload (JSON), fetched_at

3.2 Data Sources

The models learn from `ProductionRecord`, which is fed by three sources:

- **Synthetic history** (2,000 rows): generated from the real catalogue, as described in Section 3.3.
- **Live records** (3 rows today): every production task that reaches the status *Completed* is copied automatically into the history with its real duration and outcome (a Django `post_save` signal). The three tasks completed in the midterm data were imported during setup.
- **External pushes**: another system can send a finished batch with `POST /api/v1/records/`.

3.3 Synthetic Data Generation

The live database holds only a handful of tasks, far too few to train a model. A synthetic history was therefore generated, but in a way that keeps it **faithful to the company**: every synthetic batch uses a real product with its real Bill of Materials, and a real artisan with his or her real department and contract. Only the dates, quantities and outcomes are simulated.

Each batch first receives an **engineering standard duration**, the time a typical artisan would need under normal conditions:

```
hours_per_unit = category_hours x (1 + 0.06 x bom_items)
```

```
                                x (1 + 0.015 x min(bom_qty_per_unit, 20))
standard_days  = max(0.5, quantity x hours_per_unit / 8)
category_hours: Shoes 4.0 | Bags 3.0 | Belts 1.2 | Folders 1.0 | Accessories 0.6
```

The actual duration is then produced by a transparent causal process. Each factor corresponds to something a production manager would recognise:

```
actual_days = standard_days
    x artisan skill          hidden factor per artisan, U(0.75, 1.20)
    x contract factor        Permanent 0.95 | Temporary 1.05 | Intern 1.20 | none 1.10
    x (1 + 0.08 x workload)  workload = other active tasks of the artisan
    x 1.18                   if the artisan works outside his/her speciality
    x 1.15                   if the batch starts in the Nov/Dec peak season
    x log-normal noise       sigma = 0.18
    + restock wait           if stock is short: 2 + 10 x (1 - coverage) + U(0, 3)

allotted_days = ceil(standard_days x U(0.90, 1.75))
delayed       = actual_days > allotted_days
```

Table 5: *Synthetic data parameters*

Parameter	Value
Number of batches / history window	2,000 batches over the last 540 days (about 18 months)
Random seed	42 (the history is reproducible)
Batch size per category	Bags 2–25, Shoes 4–40, Belts 5–60, Folders 5–50, Accessories 10–80
Artisan choice	55 % from the department matching the category, otherwise any artisan
Workload at start	Exponential, mean 1.4 tasks (2.2 in peak season), capped at 8
Stock coverage	Log-normal around 2.7× the requirement; 12 % of batches start with a shortage (coverage 0.05–0.98)
Defects	Binomial, about 1.5 % per unit, higher for complex products and outside the speciality
Resulting delay rate	34.4 % of batches late (690 of 2,003); on-time rate 65.6 %

Why a known causal process? Because the rules that generate the data are written down, the team can check that the model has learned the right things. Chapter 5 shows that it does: the factors it considers most important are exactly the ones built into the process (stock coverage, planning slack, workload and artisan history).

3.4 Dataset Overview

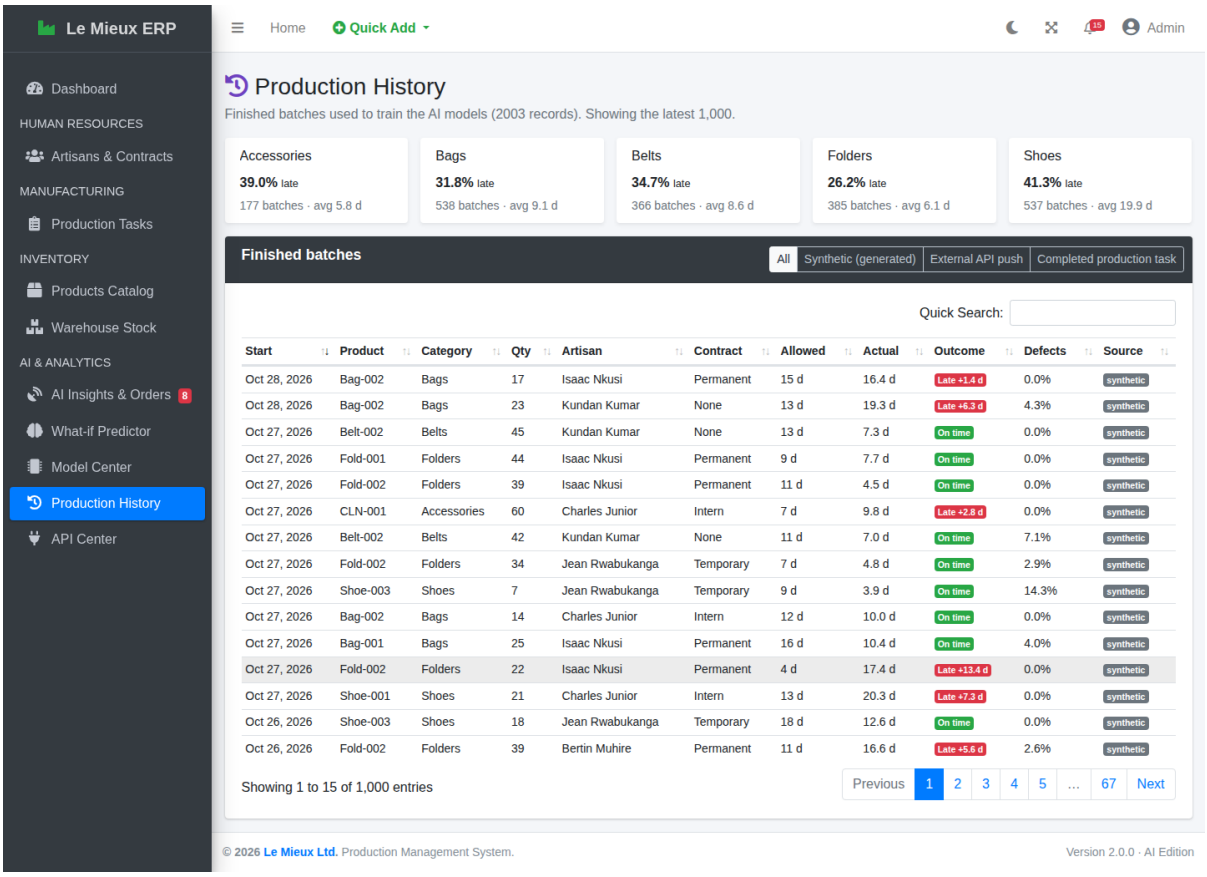


Figure 2: *Production History page: the training data with delay rate per category*

4. REST API

4.1 Design Principles

- **Versioned base path** `/api/v1/` so that future changes do not break existing clients.
- **JSON everywhere**, including errors: an API client never receives an HTML error page.
- **One response envelope**: success `{"status": "ok", "data": ..., "meta": {...}}`; error `{"status": "error", "error": {"code", "message", "details"}}`.
- **Correct HTTP status codes**: 200, 201 Created, 400 validation error, 401 not authenticated, 403 forbidden, 404 not found, 405 method not allowed, 409 conflict (training already running or failed), 503 AI engine unavailable.
- **Same business logic as the web pages**: the API calls the same service functions, so validation and predictions are identical whichever channel is used.
- **No sensitive data**: salaries are deliberately excluded from `/employees/`.

4.2 Authentication and Authorisation

Two authentication methods are accepted, checked in this order:

- **API key** in the header `X-API-Key: <key>` (or `Authorization: Api-Key <key>`). Keys are random 43-character strings prefixed `lm_`. Only a SHA-256 hash is stored, so a stolen database does not reveal usable keys. The plain key is shown once, at creation. A key acts with the permissions of its owner and can be revoked at any time.
- **Browser session** of a logged-in user, used by the dashboard's AJAX calls. For POST requests the CSRF token is verified manually, so session-based writes keep Django's CSRF protection even though the API views accept key-based clients.

Authorisation is simple and safe: every GET endpoint needs an authenticated user (except the public `/` and `/health/`), and every POST that changes data needs a **staff** user. `POST /predict/` is the exception, because it stores nothing; any authenticated user may call it.

4.3 Endpoint Reference

Table 6: *REST API endpoints*

Method	Path	Description	Access
GET	/api/v1/	API index with the list of endpoints	public
GET	/api/v1/health/	Service status and active model versions	public
GET	/api/v1/products/	Products with BOM, production cost and margin (?category=)	read
GET	/api/v1/materials/	Raw materials, stock value, low-stock flag (?low_stock=1)	read
GET	/api/v1/employees/	Artisans with department and contract, no salary (?active=1)	read
GET	/api/v1/tasks/	Production tasks with progress and overdue flag (?status=)	read
GET	/api/v1/tasks/<id>/prediction/	AI assessment of one task	read
GET	/api/v1/records/	Training data, paginated (?page, page_size ≤ 5000, source, since)	read
POST	/api/v1/records/	Push a finished batch from an external	staff

Method	Path	Description	Access
		system	
GET	/api/v1/orders/	Incoming orders with predictions (?status, risk, since_id, limit)	read
POST	/api/v1/orders/	Ingest a new order; it is scored instantly	staff
GET	/api/v1/orders/<id>/	One order with its full prediction	read
POST	/api/v1/predict/	Stateless what-if prediction (nothing stored)	read
GET	/api/v1/model/	Active models, metrics and version history	read
POST	/api/v1/model/retrain/	Retrain and redeploy both models	staff
GET	/api/v1/dashboard/summary/	KPIs and chart series	read
GET	/api/v1/forecast/materials/	Material demand forecast	read
GET	/api/v1/external/fx/	Open API exchange rate + inventory value in USD (?refresh=1)	read

4.4 Example: Ingesting an Incoming Order

```
curl -X POST http://127.0.0.1:8000/api/v1/orders/ \
  -H "X-API-Key: lm_xxxxxxxx" -H "Content-Type: application/json" \
  -d '{"product_code": "Bag-002", "quantity": 5, "due_date": "2026-11-20",
       "customer": "Kigali Heights Boutique", "channel": "Online"}'

HTTP/1.1 201 Created
{"status": "ok",
 "data": {"reference": "ORD-261029-0009", "product_code": "Bag-002", "quantity": 5,
   "status": "New",
   "prediction": {"delay_probability": 0.0329, "risk_level": "Low",
     "predicted_days": 4.1, "predicted_completion": "2026-11-03",
     "expected_delay_days": 0.0, "recommended_artisan": "Bertin Muhire",
     "stock_ok": true, "shortages": [],
     "recommendations": ["On track: the current plan is realistic. ..."],
     "model_version": "delay_classifier-v1+duration_regressor-v1"}},
 "meta": {"generated_at": "2026-10-29T17:39:07Z"}}
```

A request with mistakes returns every problem at once, so the client can fix them in one go:

```
HTTP/1.1 400 Bad Request
{"status": "error",
 "error": {"code": "validation_error", "message": "The order is not valid.",
   "details": {"product_code": "Unknown product code 'NOPE'.",
               "quantity": "Must be an integer between 1 and 10000.",
               "due_date": "This field is required (YYYY-MM-DD)."}}}
```

4.5 Open API Integration (Exchange Rates)

The ERP also **consumes** a public Open API: the ExchangeRate-API open access endpoint `https://open.er-api.com/v6/latest/USD`, which needs no key. The rate is used to show the value of the inventory in US dollars on the dashboard, which is useful for export pricing. Because an ERP must never stop working when the internet is down, the client follows a four-step policy:

1. serve the cached value if it is less than 6 hours old;
2. otherwise call the Open API with a 5-second timeout;

3. if the call fails, serve the last good value stored in the database (`ExternalDataSnapshot`);
4. if there has never been a good value, serve the fallback rate from the settings (1,450 RWF), clearly labelled *offline*.

The dashboard loads the rate asynchronously, so a slow network never delays the page. The setting `AI_EXTERNAL_APIS_ENABLED = False` switches the call off completely.

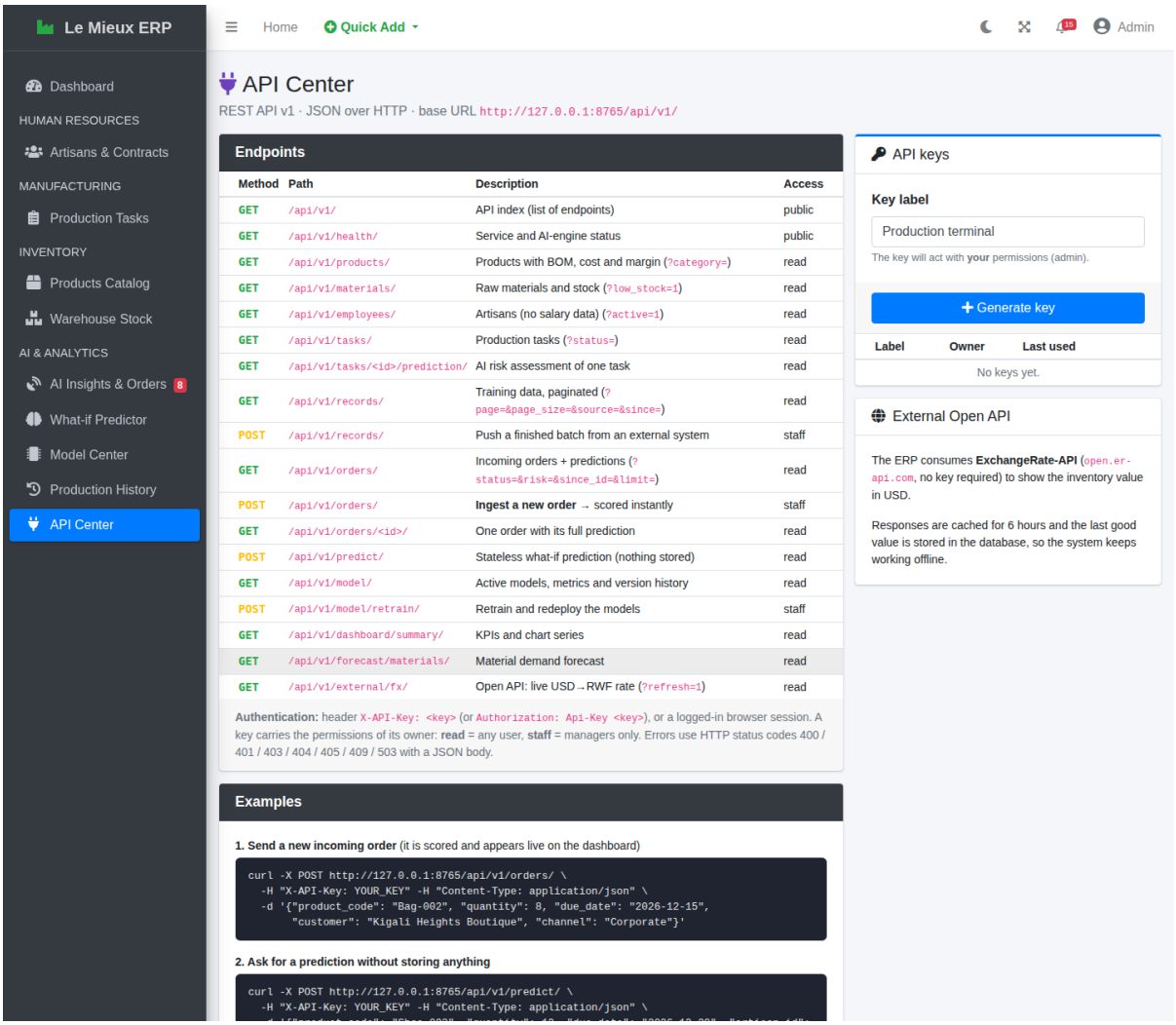


Figure 3: *API Center: endpoint documentation, examples and API key management*

5. Machine Learning

5.1 Problem Formulation

Two complementary questions are answered for every job (a product, a quantity, a due date and an artisan):

Table 7: *The two prediction tasks*

Model	Type	Target	Business question
Delay-risk classifier	Binary classification	delayed = actual_days > allotted_days	How likely is this job to be late?
Duration regressor	Regression	actual_days	How many days will this job really take, and when will it finish?

5.2 Data Acquisition through the API

To satisfy the requirement that the model is trained on data read from an API, the training pipeline never queries the training table directly. It requests `GET /api/v1/records/` page by page (1,000 records per page) and works only with the JSON it receives. Two modes are available:

- **In-process** (default): the request goes through Django's complete request/response stack (URL routing, middleware, authentication, serialisation) using an internal read-only service account, `ml-service`, which has an unusable password and cannot log in. No running server is needed.
- **Over HTTP**: `python manage.py train_models --api-url http://127.0.0.1:8000 --api-key <key>` reads from a running server exactly as an external client would. This mode was tested successfully while the server was serving users; the server picked up the new model version without a restart.

5.3 Feature Engineering

Eighteen features are built by one module, `analytics/ml/features.py`, which is used by the training pipeline and by live inference. Using the same code for both guarantees that the model sees in production exactly the kind of input it saw during training.

Table 8: *Model features*

Feature	Description
category	Product category (one-hot encoded)
quantity	Batch size in units
unit_price, material_cost_per_unit	Selling price and BOM material cost per unit (RWF)
bom_items, bom_qty_per_unit	Number of distinct materials and total material quantity per unit
standard_days	Engineering standard duration (Section 3.3)
allotted_days	Days allowed by the manager (due date − start date)
slack_ratio	allotted_days / standard_days: how generous the plan is
required_rate	Units per day needed to meet the due date
artisan_contract	Permanent, Temporary, Intern or None (one-hot)

Feature	Description
dept_match	1 if the artisan's department matches the product category
artisan_workload	Other active tasks of the artisan
artisan_hist_ontime	Artisan's historical on-time rate
artisan_hist_efficiency	Artisan's historical speed (standard / actual days)
stock_coverage	Minimum of stock / requirement over the BOM, capped at 5
month, peak_season	Start month and November–December flag

5.4 Preventing Data Leakage

The two artisan-history features could easily leak the answer: if a batch's own outcome were included in the artisan's on-time rate, the model would appear excellent in testing and fail in practice. The pipeline therefore computes them **only from batches that finished before the current batch started** (using sorted end dates and binary search), and requires at least three earlier batches; otherwise neutral defaults are used. An automated test checks this rule.

5.5 Evaluation Method

A random split would let the model learn from the future. The data is therefore split **by time**: the oldest 80 % of batches (1,602, up to 18 July 2026) are used for training and the newest 20 % (401) for testing, which reproduces how the model is used in reality: trained on the past, applied to the future. After selection, the winning algorithm is refitted on all records before deployment.

5.6 Algorithm Comparison

Three algorithms of increasing flexibility were compared for each task, all as scikit-learn pipelines with a `DictVectorizer` for one-hot encoding (plus a `StandardScaler` for the linear models). The classifier is selected by ROC-AUC, which measures how well it ranks risky jobs above safe ones regardless of the threshold; the regressor is selected by mean absolute error (MAE), which is expressed in days and easy to explain.

Table 9: *Delay-risk classifier: algorithm comparison on the test set*

Classifier	Accuracy	F1	ROC-AUC
Logistic Regression (balanced)	0.778	0.683	0.852
Random Forest (250 trees)	0.838	0.764	0.925
Gradient Boosting (250 stages) — selected	0.855	0.779	0.934

Table 10: *Duration regressor: algorithm comparison on the test set*

Regressor	MAE (days)	RMSE (days)	R^2
Ridge Regression	2.727	3.736	0.807
Random Forest (250 trees)	1.754	2.810	0.891
Gradient Boosting (300 stages) — selected	1.698	2.783	0.893

5.7 Results of the Deployed Models

Delay-risk classifier (Gradient Boosting, v3). On the 401 unseen batches it reaches an accuracy of 85.5 %, a precision of 76.7 %, a recall of 79.1 % and a ROC-AUC of 0.934. In practice, when the system says *late*, it is right about three times out of four, and it catches about four out of five jobs that really end late.

Table 11: *Confusion matrix of the classifier (test set, threshold 0.5)*

	Predicted on time	**Predicted late**
Actually on time (272)	241 correct	31 false alarms
Actually late (129)	27 missed	102 caught

Duration regressor (Gradient Boosting, v3). The mean absolute error is **1.70 days**, the R^2 is 0.893, the median relative error is 12.1 %, and 68.8 % of predictions fall within ±20 % of the real duration. For batches that typically last 3 to 25 days this is precise enough to promise realistic dates to customers.

5.8 What the Models Learned

Feature importance (Figure 4) shows that the models rely on meaningful factors. For delay risk the most important are **stock coverage** (38.8 %: a shortage means waiting for a restock), **slack ratio** (23.9 %: how generous the plan is), **artisan workload** (8.9 %), the artisan's **historical on-time rate** (7.7 %) and **speed** (5.2 %). For duration, the **engineering standard** dominates (78.1 %), followed by stock coverage (11.1 %) and workload (2.9 %). These are exactly the mechanisms built into the data, which confirms that the models learned the causes of delay rather than accidental patterns.

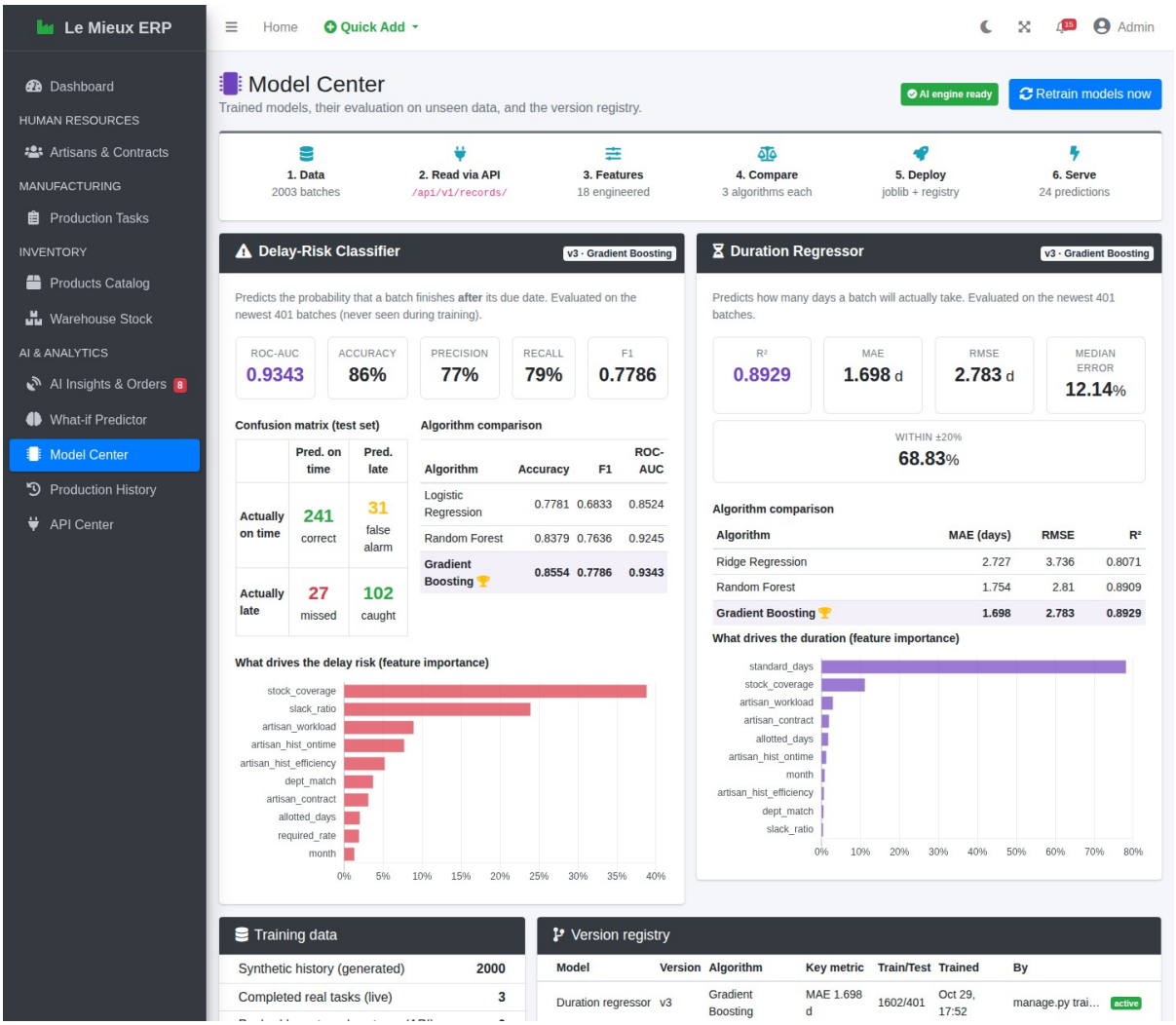


Figure 4: *Model Center: metrics, confusion matrix, algorithm comparison and feature importance*

5.9 Limitations of the Models

- The models are trained mostly on **synthetic** data. They reproduce the assumptions of Section 3.3, not yet the real behaviour of the factory. The continuous-learning loop (Section 6.4) is designed to correct this progressively: as real tasks are completed, they enter the training data and the next retraining reflects reality more.
- The factory has five artisans; predictions for a newly hired artisan rely on contract and department until three batches have been recorded for him or her.
- Probabilities are not formally calibrated. They are suitable for ranking and for the Low / Medium / High bands, but should not be read as exact frequencies.

6. Deployment in Django

6.1 Model Registry and Artefacts

Each training run produces a new **version** of each model. The fitted pipeline is saved with joblib as `analytics/ml_models/<name>_v<N>.joblib`, together with the scikit-learn version used. A row in `MLModelVersion` records the algorithm, the metrics, the comparison of all candidates, the feature importances, the size of the training and test sets, the data source and who trained it. Only one version per model is active; older versions remain in the registry as history, which makes it easy to see how the models evolve.

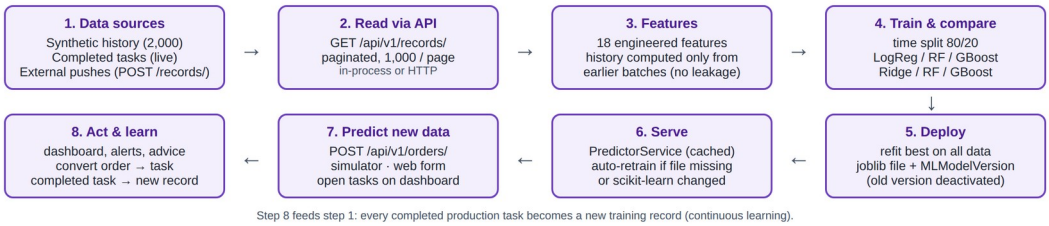


Figure 5: *Machine-learning lifecycle, from data to live predictions and back*

6.2 Inference Service

`PredictorService` (`analytics/ml/service.py`) is the single entry point for predictions. It loads the active versions once and keeps them in memory, protected by a lock so that several requests can be served safely. When a new version is activated (even by a training run started from another terminal), the service notices the change on the next prediction and loads it, without restarting the server. Predictions for several artisans are computed in **one batch call**, which keeps the scoring of an order fast.

6.3 Self-Healing

Machine-learning files are fragile when a project is moved between computers: a model saved with one scikit-learn version may not load with another. The service therefore checks every model it loads. If the file is missing, cannot be read, or was produced by a different scikit-learn version, it **retrains automatically** from the data in the API (about 10 seconds) and continues. If the database contains no training data at all, it first generates the synthetic history. A fresh installation therefore works on the first click, and the shipped models never become a point of failure. Both situations are covered by automated tests.

6.4 Continuous Learning

When a manager marks a production task as *Completed*, a signal handler creates a `ProductionRecord` with the real start date, end date, duration and outcome (source *live*). The handler runs only once per task and never blocks the save: any error is logged and ignored, so analytics can never break the core workflow. The next retraining automatically includes these real records.

6.5 Retraining

Models are retrained with the **Retrain models now** button of the Model Center (managers), `python manage.py train_models` or `POST /api/v1/model/retrain/`. A lock prevents parallel runs; a run takes about 8 seconds.

7. Real-Time Prediction of Incoming Data

7.1 The Ingestion Pipeline

New data enters the system as **incoming orders**: production requests from the storefront, corporate clients, the online shop or export partners. An order can arrive through the API (`POST /api/v1/orders/`), through the order simulator, or through the What-if Predictor (*Submit as incoming order*). In every case the same function, `ingest_order`, is used. Figure 6 shows its steps.

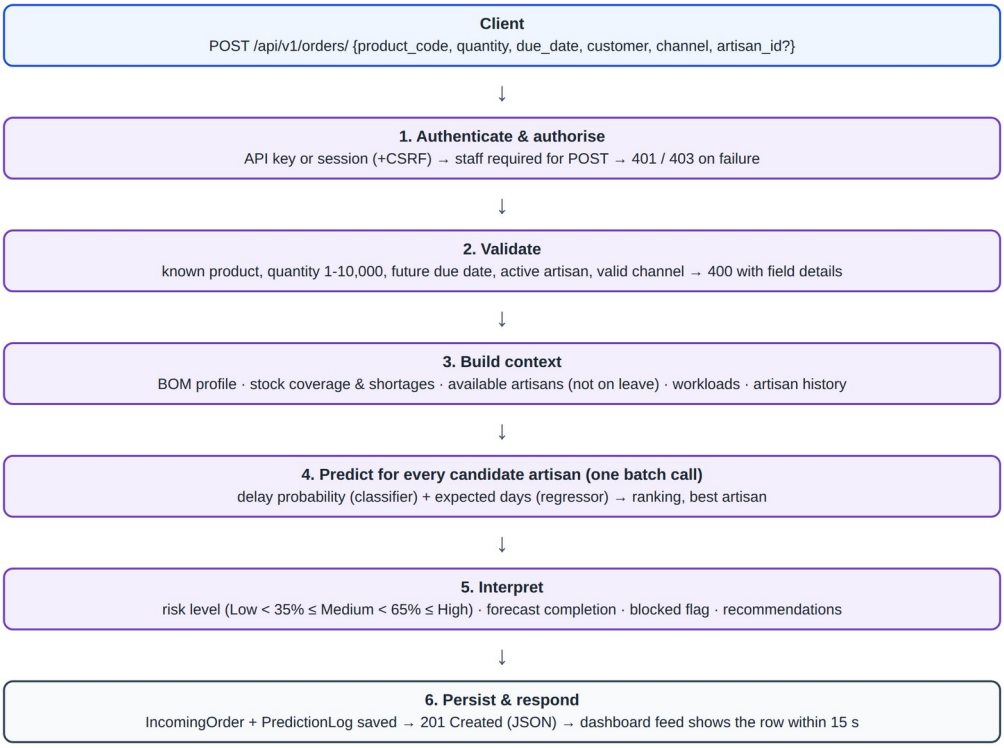


Figure 6: *Real-time ingestion and scoring of an incoming order*

7.2 Risk Levels

Table 12: *Risk levels*

Risk level	Delay probability	Meaning for the manager
Low	below 35 %	The plan is realistic; standard scheduling is enough.
Medium	35 % to 65 %	Watch the job; consider a more suitable artisan or a little more time.
High	65 % and above	The job is likely to be late; act before accepting it (reassign, extend, restock or split).
Overdue	—	An open task already past its due date (no prediction needed).

7.3 Artisan Recommendation

For each order the system lists the active artisans who are **not on approved leave** during the job (the same rule the original system enforces when a task is assigned), computes the delay probability and the expected duration for each of them, and ranks them from the safest to the

riskiest. The first one becomes the recommended artisan. If the customer or the manager requested a specific artisan, the prediction is made for that person, and the system points out when another artisan would reduce the risk noticeably.

7.4 Stock Check and Recommendations

The Bill of Materials is compared with the warehouse. When stock is insufficient, the job is marked **Blocked until restocked**, because the original stock rule would refuse to create the task, and the missing quantity of each material is listed. The prediction already includes the expected restock wait. The system then writes plain-language recommendations from simple rules:

- restock a material that is short, with the quantity missing;
- assign the job to another artisan when this lowers the risk by more than 10 points;
- rebalance work when the chosen artisan already has three or more active tasks;
- allow more days (about 10 % above the expected duration) or split the batch when the expected duration exceeds the time allowed;
- otherwise confirm that the plan is realistic.

7.5 Scoring Open Production Tasks

The same models assess every open production task on the dashboard, on the AI Insights page, on each task's detail page and through `GET /api/v1/tasks/<id>/prediction/`. For an existing task, the materials were already reserved when it was created, so stock is not treated as a risk. Overdue tasks are flagged directly. For tasks in progress, the system also projects the finishing date from the actual production pace (units produced per day so far).

7.6 From Prediction to Action

A manager can turn an order into a real production task with one click (**Create task**). The standard task form opens pre-filled with the product, quantity, due date, the recommended artisan and a description summarising the prediction. Saving it goes through **all the original validation rules**: stock availability, leave, state machine and due date. If they pass, the materials are deducted exactly as before and the order is marked *Converted* with a link to the task. Orders that are not accepted can be dismissed.

7.7 Simulating a Stream of Orders

To demonstrate real-time behaviour, `python manage.py simulate_orders` generates realistic orders (real products, typical quantities, customers and channels) and sends them one by one. With `--api-url` and `--api-key` it posts them over HTTP to the running server like an external system; without these options it uses the same pipeline in-process. The terminal shows each order's risk in colour, and the dashboard's live feed displays the new rows within 15 seconds. Managers can also press **Simulate orders** on the dashboard or the AI Insights page.

```
$ python manage.py simulate_orders --count 2 --interval 1       --api-url http://127.0.0.1:8000
--api-key lm_...
Streaming incoming orders (HTTP -> http://127.0.0.1:8000). Press Ctrl+C to stop.
ORD-261029-0013 Belt-002 x32 due 2026-11-08 -> Low  13% ~8.1 d  artisan: Bertin Muhire
ORD-261029-0014 Shoe-002 x24 due 2026-11-17 -> High 84% ~23.9 d artisan: Isaac Nkusi
2 order(s) sent.
```

8. User Interface and Dashboard

All new pages use the same AdminLTE layout, colours and components as version 1.0, work in light and dark mode, and are reached from a new sidebar section, **AI & ANALYTICS**. Every list has a clear empty state, and every AI widget shows a short explanation when the AI engine is unavailable instead of an error page.

8.1 Enhanced Dashboard

The original dashboard is kept in full: the alert banner, the four summary boxes, the recent tasks and the active workload. A new KPI row was added under the summary boxes: **inventory value** in RWF and USD (Open API), **on-time rate** (overall and last 90 days), **high-risk open tasks** and **AI model quality**.

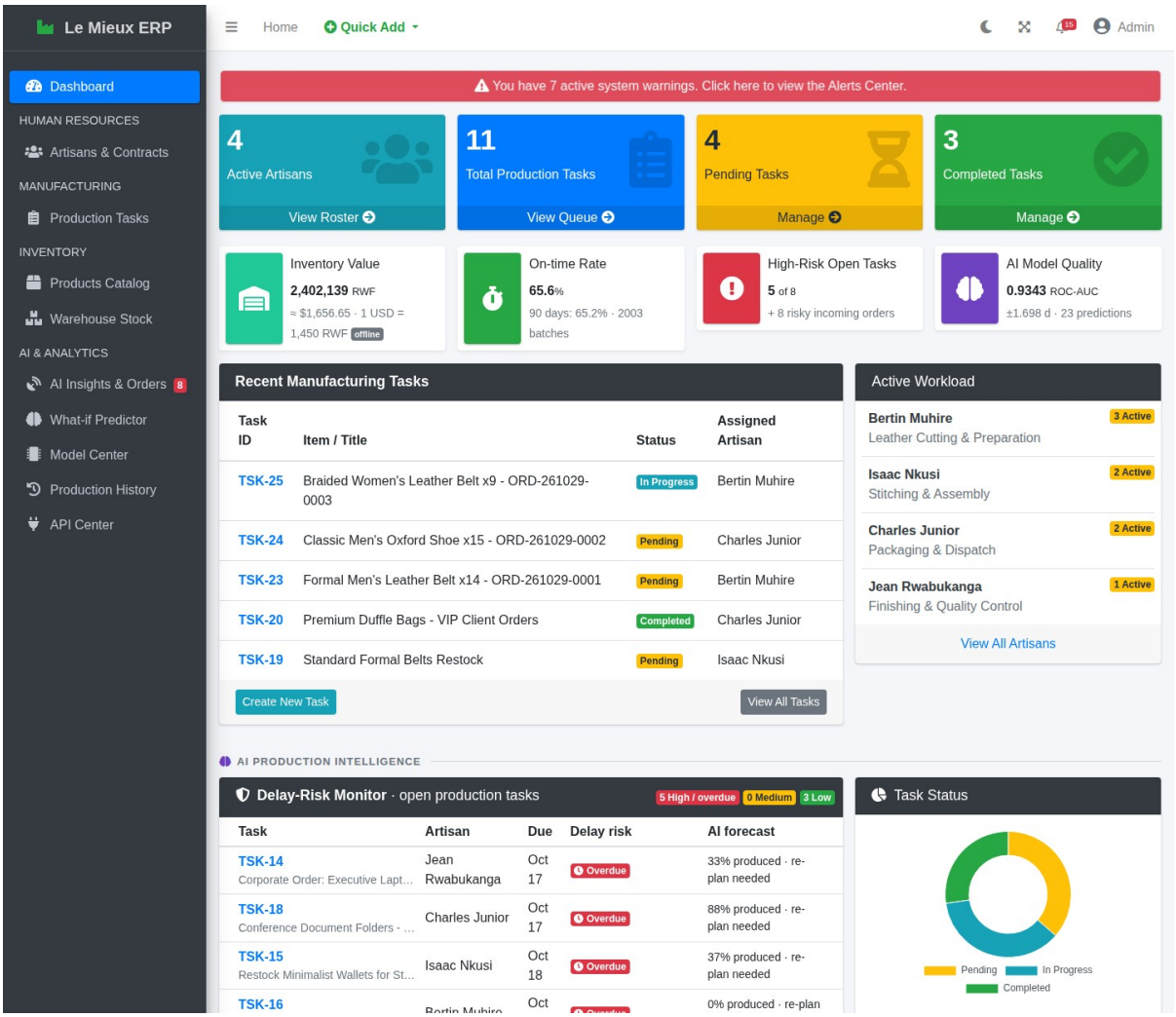


Figure 7: *Dashboard (top): original widgets and the new KPI row*

Below, the *AI Production Intelligence* section adds the **Delay-Risk Monitor** (open tasks ranked by risk, with a risk bar and the forecast completion date), a **task status** chart, the **Live Incoming Orders** feed (new orders appear automatically every 15 seconds and are highlighted), the **Material Demand Forecast** (projected stock if every new order is accepted), the **monthly production output and delay rate** over 12 months, and the **delay rate by category**.

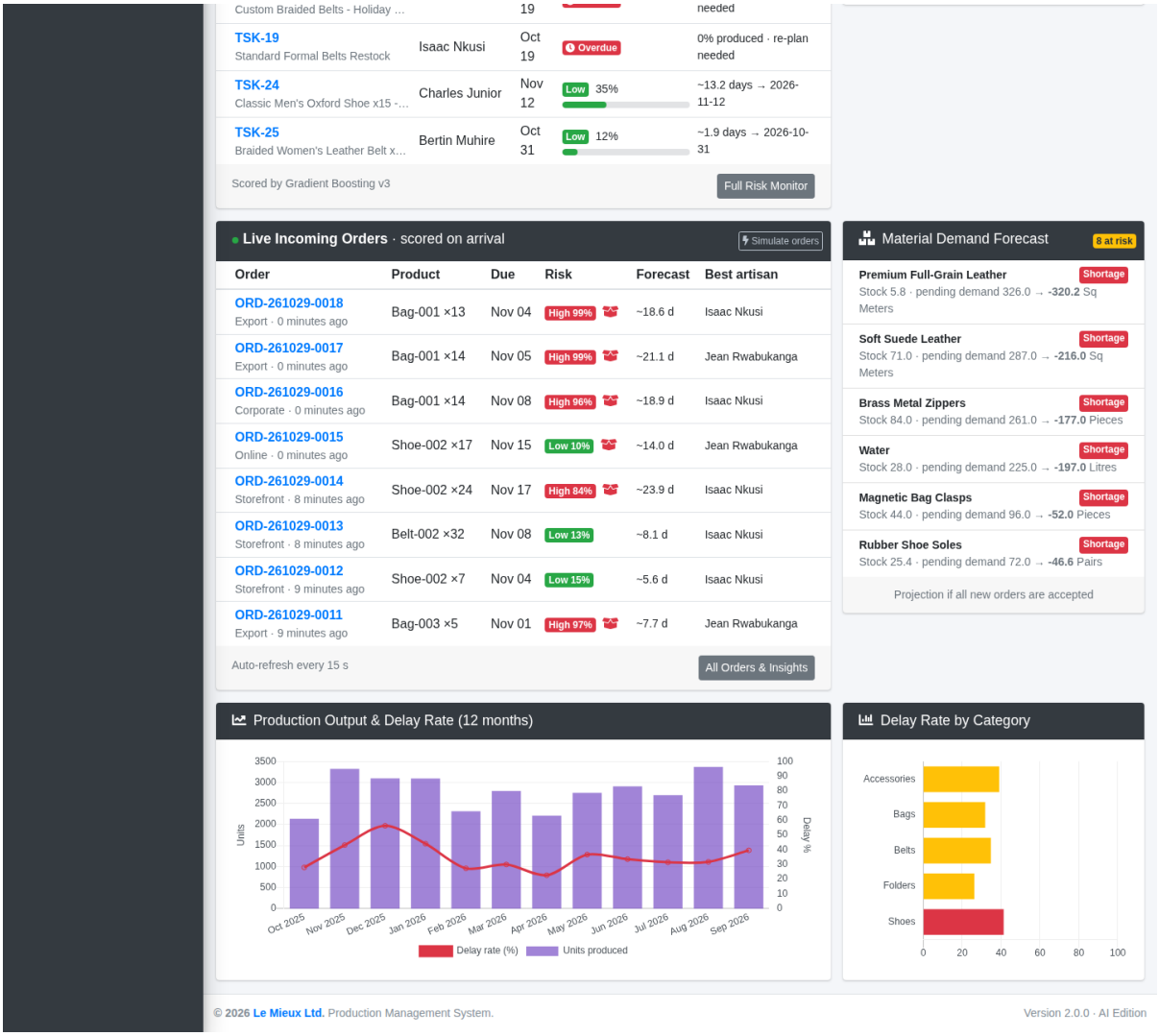


Figure 8: *Dashboard (bottom): AI Production Intelligence section*

8.2 AI Insights and Orders

This page is the inbox of incoming orders. Counters show new, high-risk, converted and total orders. The list can be filtered by status and risk and shows, for each order, the risk with a bar, the expected duration and finish date, the requested or recommended artisan and the stock status. Managers can view, convert or dismiss each order and simulate new ones. Below the list are the full risk monitor of open tasks, with advice, and the detailed material demand forecast.

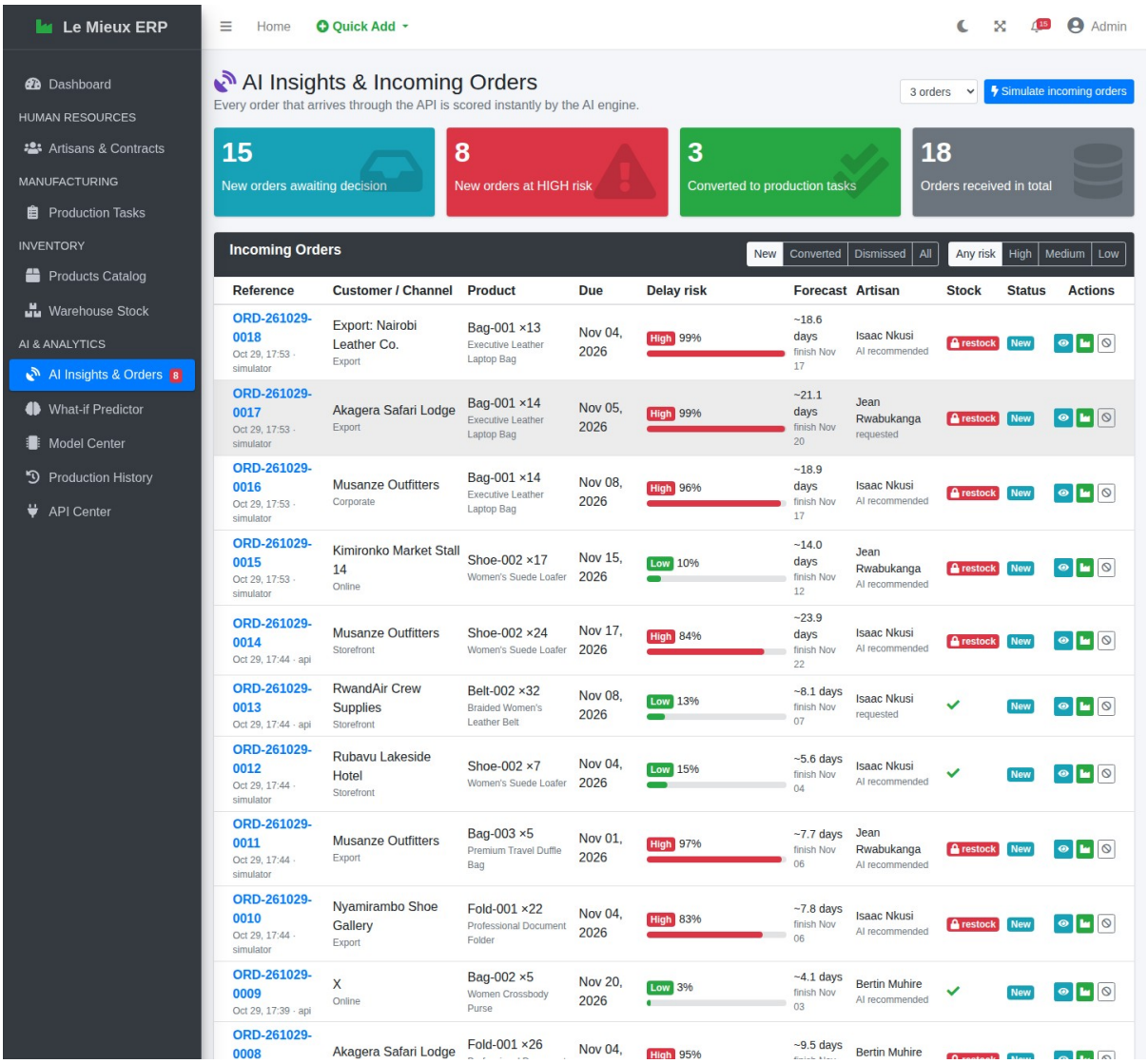


Figure 9: *AI Insights & Orders page*

8.3 Order Detail

The order page shows the order on the left and the prediction made at its arrival on the right: a risk gauge, the expected duration, the time allowed, the forecast completion and the expected lateness, the recommendations, the stock check with the missing quantities, the ranking of all artisans and, for transparency, the exact feature vector given to the model.

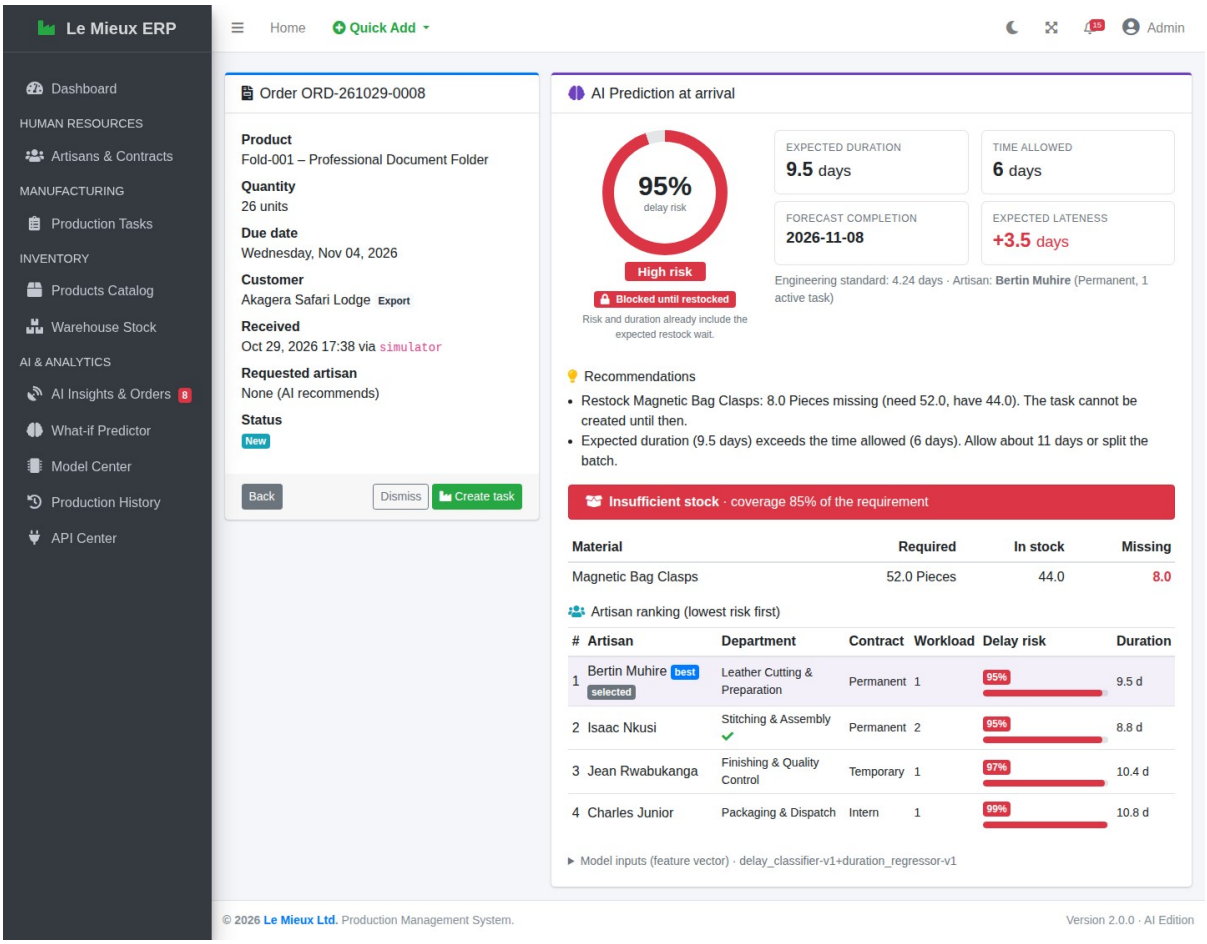


Figure 10: *Order detail with the prediction made at arrival (a job blocked by a stock shortage)*

8.4 What-if Predictor

Managers and viewers can test a job before committing to it: choose a product, a quantity and a due date, optionally an artisan, and click **Predict outcome**. Nothing is stored except an audit-log entry. A manager can then submit the same job as an incoming order.

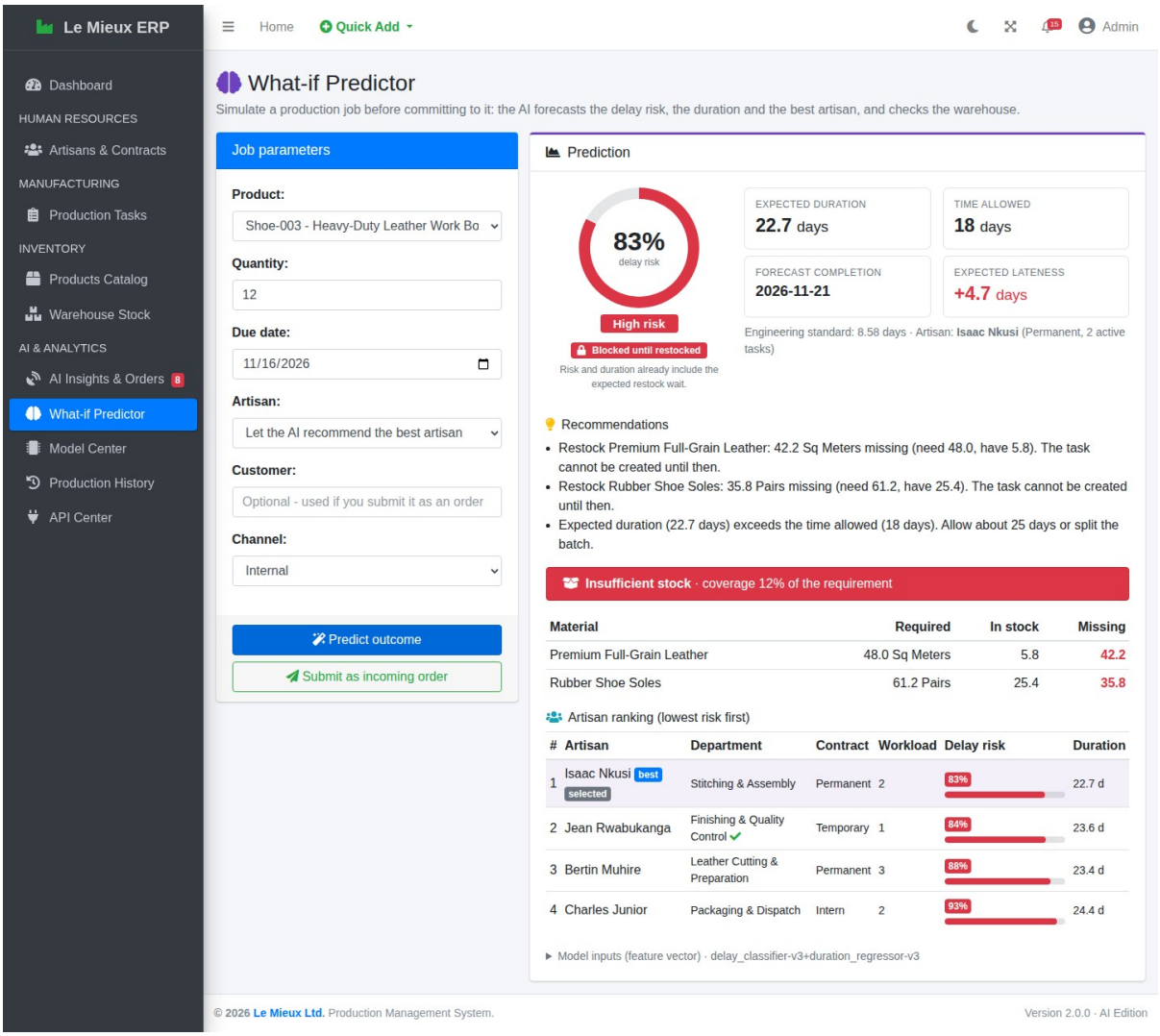


Figure 11: *What-if Predictor*

8.5 Model Center

The Model Center explains the AI to managers: the six-step pipeline, the metrics of each active model on unseen data, the confusion matrix, the algorithm comparison, the feature importance charts, the training data by source, the version registry and the audit log of the latest predictions. Managers can retrain the models with one click (Figures 4 and 12).

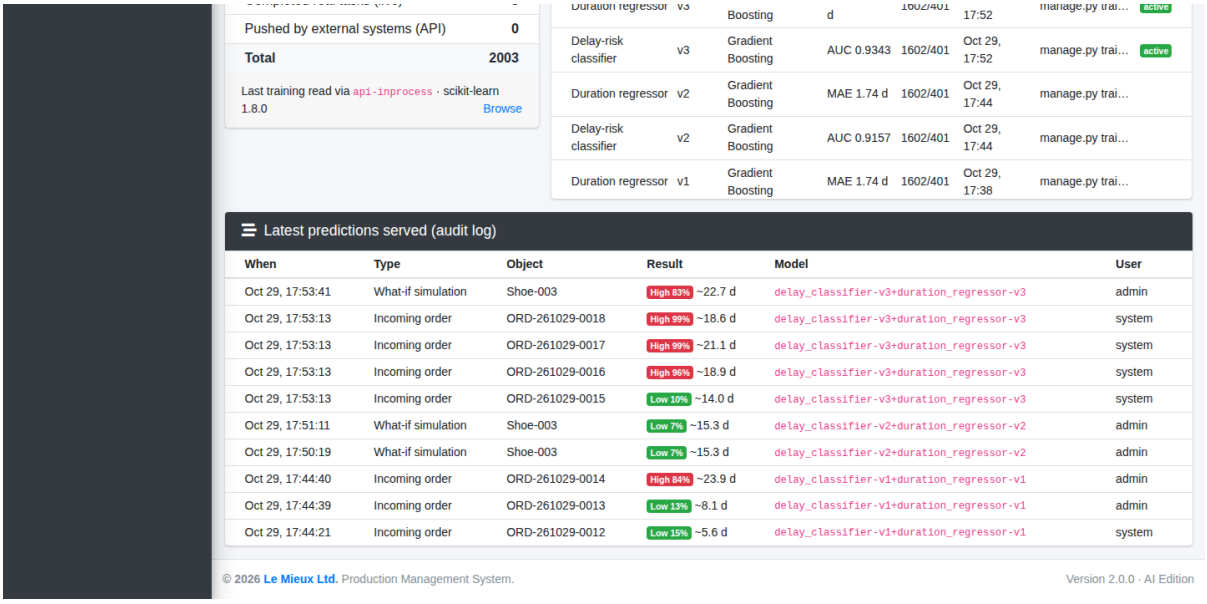


Figure 12: *Model Center (lower part): training data, version registry and prediction audit log*

8.6 Task Detail: AI Risk Assessment

Every production task page now has an *AI Risk Assessment* card with the delay probability, the expected duration, the days allowed, the forecast finish and the recommendations. For a completed task it explains that the real outcome has been added to the training data.

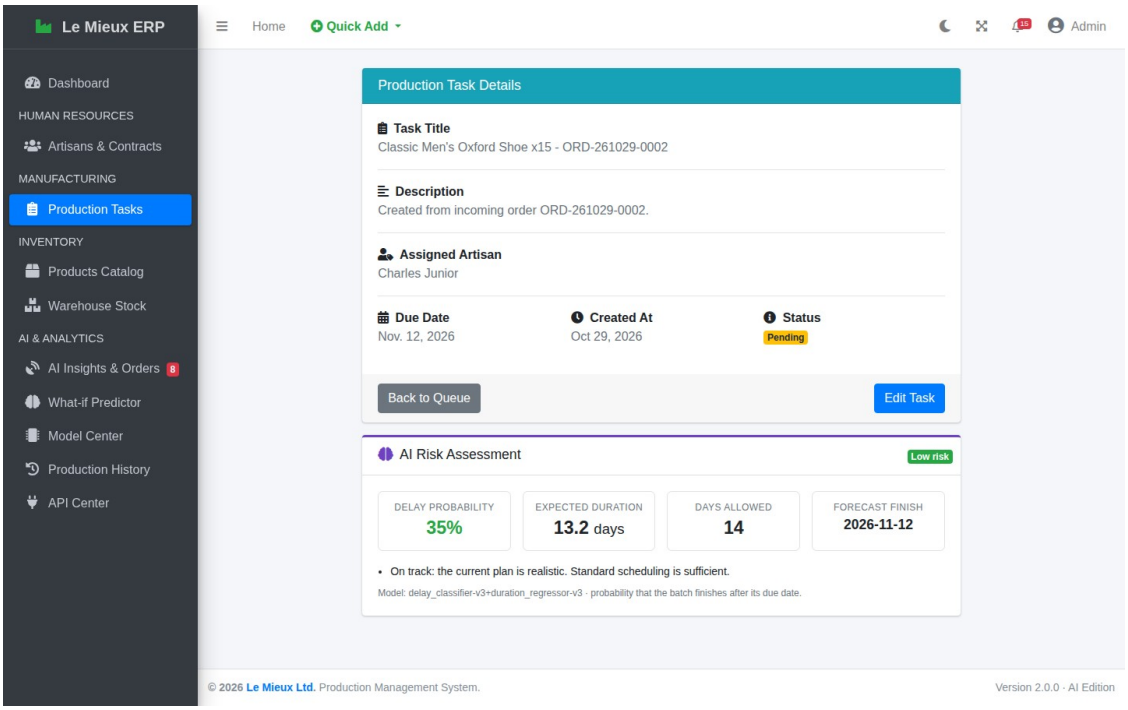


Figure 13: *Task detail with the AI Risk Assessment card*

8.7 Navigation, Alerts and Dark Mode

The notification bell now displays the real number of alerts (overdue tasks, low-stock materials and high-risk incoming orders) with a breakdown and a link to the Alerts Center. The sidebar shows a red badge next to *AI Insights & Orders* when high-risk orders are waiting. The existing dark mode applies to all new widgets and charts.

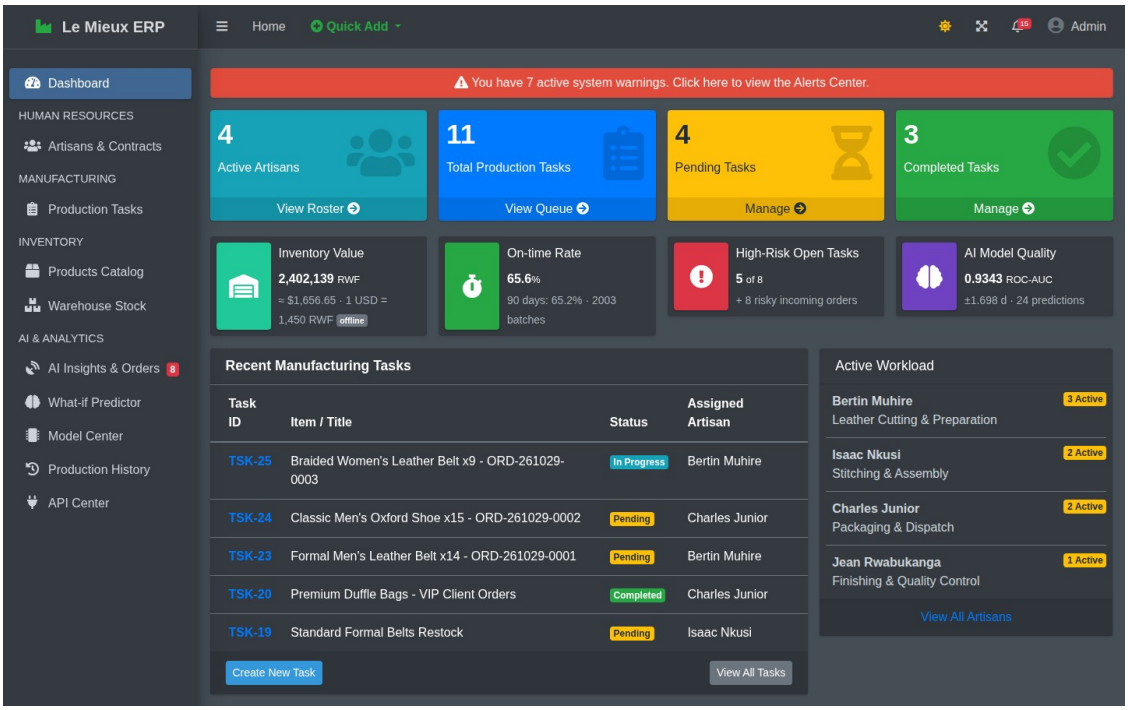


Figure 14: *Dashboard in dark mode*

9. Testing and Quality Assurance

9.1 Automated Tests

The file `analytics/tests.py` contains **40 automated tests** that build a small factory (departments, artisans, contracts, materials, products, BOMs and the three user roles), generate 400 synthetic records and train real models in a temporary folder. They run with `python manage.py test` in about 20 seconds. **All 40 tests pass.**

Table 13: *Automated test suite (40 tests, all passing)*

Group	Tests	What is verified
Machine-learning pipeline	9	Realistic synthetic data; training reads the API and registers versions with 3 candidates; minimum quality (ROC-AUC > 0.70, R^2 > 0.50); new version on retraining; no target leakage; prediction ranges and artisan ranking order; stock-shortage detection; artisans on leave excluded; self-healing when a file is missing and when the scikit-learn version differs
REST API	14	Public endpoints; authentication (no key, wrong key, header formats, revoked key); every read endpoint; salaries not exposed; pagination; staff can ingest and the order is scored and logged; viewers cannot ingest but can predict; validation errors and bad JSON; record push; CSRF on session POST; 405 handling; Open API fallback and live parsing
Workflow and pages	16	All pages for all three roles; dashboard keeps original widgets and shows the new ones; What-if page; order conversion deducts stock; simulate and dismiss; API key creation and revocation; completed task creates one training record; task risk states; alert counts
Regression of original rules	(within the 16)	Insufficient stock refused; deduction and refund of materials; new task with a past due date refused; state machine; artisan on leave cannot be assigned; access control and login requirement; the overdue-progress fix
Fresh installation	1	With no training data and no model, the first prediction generates data, trains and answers

9.2 Manual Verification

- Every URL of the system (37 original and new pages, including Django Admin) was opened as **admin**, **HR1** and **User**. No page produced a server error, and each role received exactly the access it should (pages or redirects).
- The dashboard and all new pages were opened in a real browser (headless Chromium) with no JavaScript errors, in light and dark mode.
- The HTTP path was tested end-to-end on a running server: API key creation, order streaming with the simulator, training over HTTP while the server was in use, and rejection of an invalid key (401).
- Three incoming orders were converted into production tasks through the normal form, and the warehouse stock was checked to have decreased exactly by the BOM requirement.

9.3 Defects Found and Fixed During Development

- The navbar alert count stayed at zero: the original `ProductionTask.__init__` reads its fields, so loading tasks with `.only()` caused infinite recursion. The query was changed and a comment added.

- The first synthetic history was too pessimistic (59 % of batches late). The planning slack and workload effects were recalibrated to a realistic 34 %.
- A job with a severe stock shortage could be shown as *Low risk*. The restock wait was made proportional to the size of the shortage, the model was retrained, and such jobs are now clearly labelled *Blocked until restocked*.

10. Changes to the Existing System

The extension is **additive**: no original model field, URL or business rule was removed, and the original tables were not altered. Table 14 lists every modified file; the complete list, including new files and data changes, is in `docs/CHANGES.md`.

Table 14: *Modified files*

File	Change
config/settings.py	Registered the `analytics` app and its context processor; added `ML_MODEL_DIR`, `AI_EXTERNAL_APIS_ENABLED`, `AI_FALLBACK_USD_RWF` and logging
config/urls.py	Added `/analytics/` (pages) and `/api/v1/` (REST API)
production/views.py	The dashboard view adds the AI context inside a try/except, so the original dashboard always renders
production/models.py	Bug fix: the past-due-date rule applies only to new tasks or changed due dates
templates/base.html	AI & ANALYTICS sidebar section, live alert bell, Quick Add link, stylesheet, version 2.0.0
templates/dashboard.html	New KPI row and AI section; HR links shown only to staff
production/.../task_detail.html	AI Risk Assessment card; Edit button only for staff
requirements.txt	Re-saved as UTF-8 (was UTF-16); added numpy, scipy, scikit-learn, joblib
readme.md	Rewritten for version 2.0

10.1 Corrections to Version 1.0

Table 15: *Corrections to existing behaviour*

Issue in v1.0	Correction
An overdue task could not be saved, so its progress could not be recorded, unless its due date was also moved	The rule *the due date cannot be in the past* now applies only when a task is created or its due date is changed
Read-only users saw *View Roster*, *View All Artisans* and artisan links that silently sent them back to the dashboard	These links are shown only to staff
Read-only users saw the *Edit Task* button on the task page	The button is shown only to staff
The bell icon always showed "!" whatever the situation	It shows the real number of alerts with a breakdown

10.2 Data Added to the Database

The shipped `db.sqlite3` is ready to use: it contains the 2,000 synthetic records, the 3 live records, the trained model versions (v3 active), 18 demo incoming orders and their audit log. For the demonstration, three low-risk orders were converted into production tasks **TSK-23, TSK-24 and TSK-25** through the normal workflow (materials were deducted by the original stock rules), and TSK-25 was set *In Progress* (3 of 9 units). The original database is kept unchanged in `backups/db_before_ai_extension.sqlite3`.

11. Installation and Operation Guide

11.1 Contents of the Delivered Folder

The zip file contains the complete project: source code, the ready-to-use database, the trained models, this report (PDF and DOCX), `docs/CHANGES.md`, screenshots, the database backup and the `.git` history. The Windows virtual environment (`venv/`) of version 1.0 is **not** included: it did not contain the new packages and is recreated in two commands. The new work is not yet committed to Git; it can be reviewed with `git status` and committed, for example on a branch `feature/ai-analytics`.

11.2 Requirements

- Python 3.12 or newer (3.14 supported).
- pip and an internet connection for the first installation of the packages.
- A modern web browser.

11.3 Installation Steps

Open a terminal in the folder that contains `manage.py`, then run:

```
python -m venv venv
venv\Scripts\activate            (Windows)
source venv/bin/activate         (macOS / Linux)
pip install -r requirements.txt
python manage.py migrate         (safe: the database is already migrated)
python manage.py runserver
```

Open **http://127.0.0.1:8000/** and log in with one of the accounts in Table 16.

Table 16: *Login accounts*

Username	Password	Role
admin	Admin@2026	Superuser: all features and Django Admin
HR1	Web@tech2	Manager (staff): all CRUD, orders, retraining, API keys
User	Web@tech3	Viewer: read-only pages and API, What-if Predictor

> **First start on another computer.** If the installed scikit-learn version differs from the one used to train the shipped models, the first AI page takes about 10 seconds because the models retrain themselves automatically. This happens once.

11.4 Optional Commands

Table 17: *Management commands*

Command	Purpose
python manage.py test	Run the 40 automated tests
python manage.py setup_ai [--force] [--orders 8]	Rebuild the AI module: training data, import of completed tasks, training through the API, demo orders
python manage.py generate_synthetic_data [--records 2000] [--days 540] [--seed 42] [--keep]	Regenerate the synthetic history

Command	Purpose
python manage.py train_models [--api-url URL --api-key KEY]	Train and deploy new model versions, reading the data from the API
python manage.py simulate_orders [--count 10] [--interval 3] [--api-url URL --api-key KEY]	Stream new incoming orders (--count 0 runs until Ctrl+C)
python manage.py create_api_key --user USERNAME [--name LABEL]	Create an API key (printed once)

11.5 Demonstration Script (about 5 minutes)

1. Log in as **admin**. On the dashboard, present the new KPI row, the Delay-Risk Monitor, the Live Incoming Orders, the Material Demand Forecast and the charts.
2. Click **Simulate orders**: three new orders arrive and are scored immediately.
3. Open **AI Insights & Orders**, filter on *High*, open an order and explain the gauge, the stock check and the artisan ranking. Go back, choose a *Low* risk order and click **Create task**; show that the warehouse stock decreased.
4. Open the **What-if Predictor**, choose a product, quantity and date, leave the artisan empty and click **Predict outcome**.
5. Open the **Model Center**, explain the metrics and feature importance, then click **Retrain models now**.
6. In a second terminal, create a key and stream orders over HTTP: `python manage.py create_api_key --user HR1` then `python manage.py simulate_orders --count 20 --interval 4 --api-url http://127.0.0.1:8000 --api-key <KEY>`. Watch the dashboard update by itself.
7. Optionally train over HTTP: `python manage.py train_models --api-url http://127.0.0.1:8000 --api-key <KEY>`, and log in as **User** to show the read-only view.

11.6 Configuration

Table 18: *Configuration settings*

Setting (config/settings.py)	Default	Meaning
ML_MODEL_DIR	analytics/ml_models	Folder of the model files
AI_EXTERNAL_APIS_ENABLED	True	False = never call the Open API (fully offline)
AI_FALLBACK_USD_RWF	1450.0	Rate used if the Open API has never been reachable

11.7 Troubleshooting

Table 19: *Troubleshooting*

Symptom	Cause and solution
ModuleNotFoundError: sklearn (or numpy)	Packages not installed in the active environment: activate the venv and run `pip install -r requirements.txt`
The first AI page is slow (about 10 s)	Normal once: automatic retraining for the local scikit-learn version
The exchange rate is labelled *offline*	No internet access: the last known or fallback rate is shown;

Symptom	Cause and solution
	everything else works
simulate_orders: Cannot reach http://...	The server is not running: start `python manage.py runserver`, or omit `--api-url`
API answers 401 invalid_api_key	Key mistyped or revoked: create a new key in the API Center
API answers 403 permission_denied on POST	The key belongs to a non-staff user: use a manager's key
Training failed: only N records available	Run `python manage.py generate_synthetic_data` or `setup_ai`
Start again from the original data	Copy `backups/db_before_ai_extension.sqlite3` over `db.sqlite3`, then run `migrate` and `setup_ai`

12. Security Considerations

- **API keys are hashed** with SHA-256 and shown only once; they can be revoked, and their last use is recorded.
- **Least privilege**: writes need a staff account; viewers are read-only; the internal `ml-service` account cannot log in and is not staff.
- **CSRF protection** is kept for every browser POST, including session-authenticated API calls.
- **Input validation** on every API write, with field-level errors; redirects after actions only go to safe local URLs.
- **Data minimisation**: salaries are not exposed by the API.
- **Audit trail**: every prediction is logged with its inputs, outputs, model version and user.
- **Resilience**: the AI layer is isolated with fail-safe wrappers; an analytics error can never block a production task or the dashboard.
- **Production deployment** (beyond the scope of the course): as for version 1.0, the secret key must be moved to an environment variable, `DEBUG` set to False, `ALLOWED_HOSTS` configured, HTTPS enabled and rate limiting added to the API.

13. Limitations and Future Work

- **Real data.** Replace the synthetic history progressively with real completed tasks (already automated) and schedule a periodic retraining, e.g. weekly.
- **Probability calibration.** Calibrate the classifier (Platt or isotonic) once enough real data is available.
- **Push notifications.** Replace the 15-second polling of the live feed by WebSockets (Django Channels) and e-mail high-risk alerts.
- **Demand forecasting.** Add time-series forecasting of orders per product to plan purchases ahead.
- **API ecosystem.** Generate an OpenAPI (Swagger) specification and add rate limiting and key scopes.
- **Quality model.** The records already contain defective units; a third model could predict the defect rate of a batch.

14. Conclusion

The final project turned the Le Mieux Production Management System from a system that records production into one that anticipates it. A versioned REST API now opens the ERP to other systems and feeds a machine-learning pipeline that reads its data through that API. Two models, selected after a fair comparison of three algorithms each and evaluated on data they had never seen, predict whether a job will be late (ROC-AUC 0.934) and how long it will take (average error 1.70 days). They are deployed inside Django with versioning, caching and self-healing, and they score every new incoming order the moment it arrives.

For the production manager, the result is visible on the dashboard: which open tasks are drifting, which new orders are risky and why, which artisan should take a job, and which materials will run short. Every prediction comes with understandable recommendations and can be turned into a real task through the original workflow, whose rules are unchanged and verified by automated tests. Finally, because every completed task feeds back into the training data, the system will become more accurate the longer the factory uses it.

Appendix A. New Files Inventory

Table 20: *New files*

Path	Content
analytics/models.py	6 models (Section 3.1)
analytics/ml/features.py	Domain helpers and the 18 features; leakage-safe history
analytics/ml/synthetic.py	Synthetic history and order payload generators
analytics/ml/training.py	API data acquisition, candidates, evaluation, registry
analytics/ml/service.py	PredictorService, predict_job, score_task, ingest_order
analytics/ml/insights.py	KPIs, trends, category delay, material demand forecast
analytics/ml/external.py	Open API exchange-rate client
analytics/api/*.py	Authentication decorator, serializers, 16 endpoints
analytics/views.py, urls.py, forms.py	AI Insights, order detail/convert/dismiss, simulate, predictor, model center, retrain, history, API center, key revocation
analytics/dashboard.py	Dashboard context (fail-safe)
analytics/signals.py	Completed task → training record
analytics/context_processors.py	Navbar alert counts
analytics/templatetags/analytics_tags.py	task_risk tag; risk_badge, pct, rwf, get_item filters
analytics/management/commands/	setup_ai, generate_synthetic_data, train_models, simulate_orders, create_api_key
analytics/templates/analytics/	7 templates
analytics/static/analytics/analytics.css	Styles of the new widgets
analytics/tests.py	40 automated tests
analytics/migrations/0001_initial.py	Creates the 6 new tables
docs/	Report (PDF, DOCX), CHANGES.md, screenshots

Appendix B. API Usage from Python

```
import requests

API = "http://127.0.0.1:8000/api/v1"
HEADERS = {"X-API-Key": "lm_your_key_here"}

# 1. Read the training data (paginated)
page, records = 1, []
while True:
    r = requests.get(f"{API}/records/", headers=HEADERS,
                     params={"page": page, "page_size": 1000}).json()
    records += r["data"]
    if page >= r["meta"]["pages"]:
        break
    page += 1
print(len(records), "records")

# 2. Send a new order and read its prediction
order = {"product_code": "Belt-001", "quantity": 20, "due_date": "2026-12-01",
         "customer": "Musanze Outfitters", "channel": "Corporate"}
r = requests.post(f"{API}/orders/", json=order, headers=HEADERS)
p = r.json()["data"]["prediction"]
print(r.status_code, p["risk_level"], p["delay_probability"],
      p["predicted_days"], p["recommended_artisan"])

# 3. Ask a what-if question without storing anything
r = requests.post(f"{API}/predict/", json={**order, "artisan_id": 3}, headers=HEADERS)
print(r.json()["data"]["recommendations"])
```

Appendix C. Glossary

Table 21: *Glossary*

Term	Meaning
API key	A secret string that identifies a program calling the API, with the permissions of its owner
BOM (Bill of Materials)	The list of materials and quantities needed to make one unit of a product
Feature	A measurable input given to a model, such as quantity or workload
Gradient Boosting	An algorithm that builds many small decision trees, each correcting the errors of the previous ones
MAE	Mean absolute error: the average size of the prediction error, here in days
R^2	Share of the variation of the target explained by the model (1 = perfect)
ROC-AUC	Probability that the model gives a higher risk to a late job than to an on-time job (0.5 = random, 1 = perfect)
Precision / Recall	Of the jobs predicted late, the share really late / of the jobs really late, the share predicted late
Slack ratio	Days allowed divided by the standard duration; below 1 means an unrealistic plan
Synthetic data	Data generated by a documented process that imitates reality, used when real data is insufficient
Target leakage	Accidentally giving the model information about the answer it must predict
Time-based split	Training on older data and testing on newer data, as in real use